



Python

Install

See:

<https://docs.anaconda.com/anaconda/install/>

<https://docs.anaconda.com/anaconda/user-guide/getting-started/>

Install also:

Notebook, Spyder, Orange3

Definition

- Python is an easy to learn, powerful programming language.
- It has efficient high-level data structures and a simple but effective approach to object-oriented programming.
- Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

More Info

- Python is cross platform (for many platforms).
- The standard library is huge, very useful, and is very well documented.
 - <http://docs.python.org/library/>
- Many companies make significant use of it
 - Google uses it for a lot of their internal software projects

More Info

- Created by [Guido van Rossum](#) and first released in 1991, Python's design philosophy emphasizes [code readability](#) with its notable use of [significant whitespace](#)
- Python is [dynamically typed](#) and [garbage-collected](#). It supports multiple [programming paradigms](#), including [structured](#) (particularly, [procedural](#)), object-oriented, and [functional programming](#)

Why is it called Python?

When he began implementing Python, Guido van Rossum was also reading the published scripts from [“Monty Python’s Flying Circus”](#), a BBC comedy series from the 1970s.

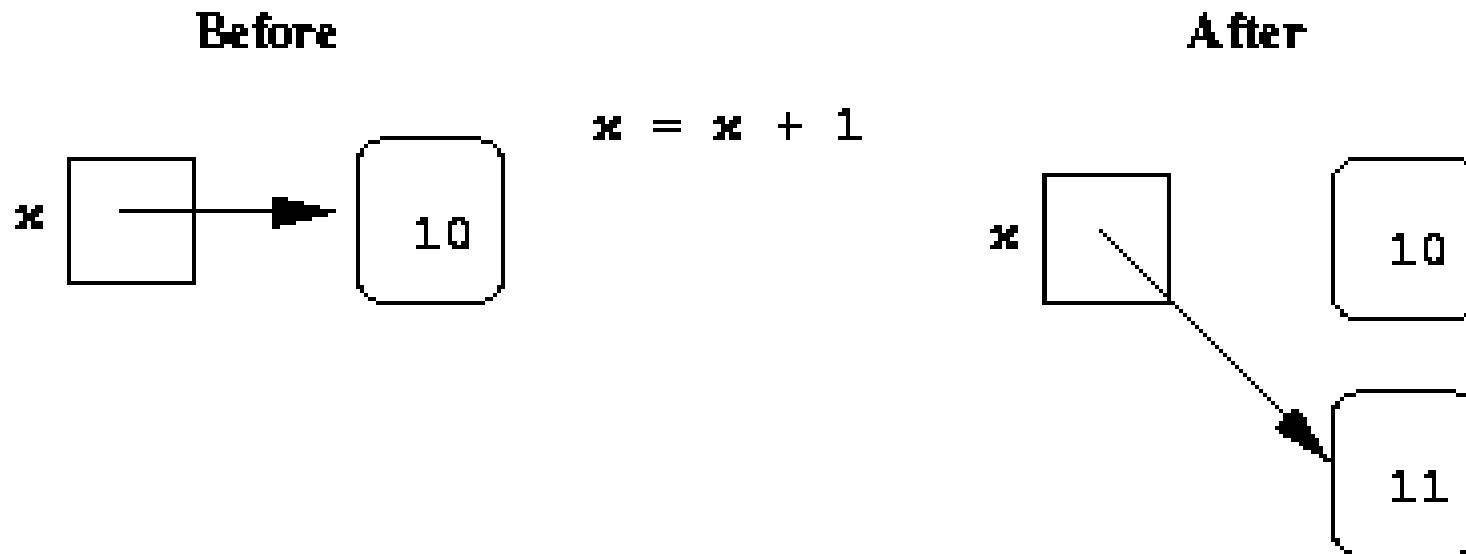


Variables and identifiers

- Besides literals, a variable can be used to hold a value.
 - A variable is a name (identifier) that is associated with a value
- An identifier is also used to name functions and modules.
- Every identifier must begin with a letter or underscore (“_”), followed by any sequence of letters, digits, or underscores.
 - Note that “—”, “.” and many other symbols are not allowed.
 - Identifiers are case sensitive.
 - Identifiers cannot be Python’s keywords.

Behind the scene

- Python does not recycle these memory locations.
 - Integer values are immutable.
- Assigning a variable is more like putting a “sticky note” on a value and saying, “this is x”.
- The memory for the old value will be “released” automatically (i.e., *garbage collection*).



List

A **list** is a linear data structure, thus its elements have a linear ordering.

- A list of names, numbers, student records, grocery, etc

You can create a list in Python by

- `myList = [1, 2, 3, 4]`
- `myGrades = ["A+", "A", "B+", "B"]`
- `myMenu = ["Sausage", "egg", "bread", "potato"]`
- `myMix = [1, "Spam ", 4, "U"]`

A **list** in Python is a mutable, linear data structure of variable length, allowing mixed-type elements.

- A list can be empty, e.g., `empty_list = []`.

Tuples

A **tuple** is an *immutable* linear data structure.

- The elements in a tuple cannot be modified.

You can create a tuple in Python by

- `myList = (1, 2, 3, 4)`
- `myGrades = ("A+", "A", "B+", "B")`
- `myMenu = ("Sausage", "egg", "bread", "potato")`
- `myMix = (1, "Spam ", 4, "U")`
- `myEmptiness = ()`

Tuples of one element must include a comma following the element, e.g., `(1, )` (instead of `(1)`).

Slicing

We can also access to a *substring/sublist/subtuple* through a process called *slicing*.

Slicing: `<string>[<start>:<end>]`

- `start` and `end` should both be integers.
- The slice contains the substring beginning at position `start` and runs up to **but doesn't include** the position `end`.
- The defaults are the beginning and the end.

Common operations for sequences

Operation		String s = 'hello' w = '!'	Tuple s = (1,2,3,4) w = (5,6)	List s = [1,2,3,4] w = [5,6]
Length	len(s)	5	4	4
Select	s[0]	'h'	1	1
Slice	s[1:4]	'ell'	(2, 3, 4)	[2, 3, 4]
	s[1:]	'ello'	(2, 3, 4)	[2, 3, 4]
Count	s.count('e')	1	0	0
	s.count(4)	error	1	1
Index	s.index('e')	1	--	--
	s.index(3)	--	2	2
Membership	'h' in s	True	False	False
Concatenation	s + w	'hello!'	(1, 2, 3, 4, 5, 6)	[1, 2, 3, 4, 5, 6]
Minimum Value	min(s)	'e'	1	1
Maximum Value	max(s)	'o'	4	4
Sum	sum(s)	error	10	10

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

The range function

A built-in **range** function can be used for generating a sequence of integers that a for loop can iterate over.

- Rather than being a function, range is actually an immutable sequence type.

Usage: `range(start, stop[, step])`

- The arguments must be integers.
- If the *step* argument is omitted, it defaults to 1.
- If the *start* argument is omitted, it defaults to 0.
- Try `range(1, 10)`, `range(-1, 5)`, `range(-1, -10, -1)`.

Actually, range does not create a sequence of integers.

- It creates a *generator function* able to produce each next item of the sequence when needed.
- This saves memory, especially for long lists.

Storing students' records

Problem: How to store student's data in a program after reading the data from a file?

A possible way is to store the data in a list.

However, a list can be indexed only by non-negative integers, but ours should be indexed by names.

- Map the names to a set of integers.
- Use the integers as indices for accessing the list.

A much better solution is to use a Python dictionary that maps the set of names (keys) directly to a set of values.

Python dictionary

A dictionary is a mutable, associative data structure of variable length.

- The key can be of any immutable type.
- The values can be of any type and are unordered.

An example:

- `dicta = {'rocky': 'CHANG Kow Chuen', ('a', 'b', 'c'): [1, 2, 3], 1: 2}`
- `divmod(x,y)` returns a tuple.
- Get the value using its index, e.g., `dicta["rocky"]`.

Dictionary operators

Operation	Results
<code>dict()</code>	Creates a new, empty dictionary
<code>dict(s)</code>	Creates a new dictionary with key values and their associated values from sequence <code>s</code> , for example, <pre>fruit_prices = dict(fruit_data)</pre> where <code>fruit_data</code> is (possibly read from a file): <pre>[['apples', .66], ..., ['bananas', .49]]</pre>
<code>len(d)</code>	Length (num of key/value pairs) of dictionary <code>d</code> .
<code>d[key] = value</code>	Sets the associated value for <code>key</code> to <code>value</code> , used to either add a new key/value pair, or replace the value of an existing key/value pair.
<code>del d[key]</code>	Remove key and associated value from dictionary <code>d</code> .
<code>key in d</code>	True if key value <code>key</code> exists in dictionary <code>d</code> , otherwise returns False.

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Dictionary methods

Python provides a number of other methods that iterate through the elements in a dictionary.

- `items()`: all the key-values pairs as a list of tuples
- `keys()`: all the keys as a list
- `values()`: all the values as a list

The objects returned by `dict.keys()`, `dict.values()` and `dict.items()` are *view objects*.

- They provide a dynamic view on the dictionary's entries, which means that when the dictionary changes, the view reflects these changes.
- They are iterable, therefore can be used in a loop.

Sets in Python

A set is collection of non-mutable objects which are elements of the set.

Set is mutable.

The elements could be of any types.

There is no order in a set.

There are no duplicate elements in a set.

`{}` is an empty set.

Set operators

Set operator		Set A = {1,2,3}	Set B = {3,4,5,6}	
membership		1 in A	True	<i>True if 1 is a member of set</i>
add		A.add(4)	{1,2,3,4}	<i>Adds new member to set</i>
remove		A.remove(2)	{1,3}	<i>Removes member from set</i>
union		A B	{1,2,3,4,5,6}	<i>Set of elements in either set A or set B</i>
intersection		A & B	{3}	<i>Set of elements in both set A and set B</i>
difference		A - B	{1,2}	<i>Set of elements in set A, but not set B</i>
symmetric difference		A ^ B	{1,2,4,5,6}	<i>Set of elements in set A or set B, but not both</i>
size		len(A)	3	<i>Number of elements in set (general sequence operation)</i>

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Function definition

The part of the program that creates a function is called *a function definition*.

```
def <name> (<formal-parameters>) :  
    <body>
```

E.g.,

```
def avg(n1, n2, n3) :  
    print((n1+n2+n3)/3)
```

- where `n1`, `n2`, `n3` are *formal parameters*.

A function is called/invoked by `<name> (<actual-parameters>)`

- E.g., `avg(1, 2, 3)`, where 1, 2 and 3 are *actual parameters*.

Functions must be defined/imported before it is called.

(Non-)Value-returning functions

The three arguments are passed to the three parameters according to their positions.

A **value-returning function** is a program routine called for its return value.

- The *return statement* of the form `return expr`, where `expr` may be any expression.

A **non-value-returning function** is called not for a returned value, but for its *side effects*.

- A **side effect** is an action other than returning a function value, such as displaying output on the screen.
- There is no return statement.

Positional and keyword arguments

A **positional argument** is an argument that is assigned to a particular parameter based on its position in the argument list.

A **keyword argument** is an argument that is specified by parameter name.

- E.g.,

```
def printNumbers(n1, n2, n3):  
    print(n1, n2, n3)
```

```
m1, m2, m3 = 10, 20, 30
```

```
printNumbers(m1, m2, m3)
```

```
printNumbers(n1=m2, n2=m3, n3=m1)
```

Local scope and local variables

Scope: places in a program where the parameters and variables are referenced.

A **local variable** is a variable that is only accessible from within a given function. Such variables are said to have **local scope**.

Formal parameters also have local scope.

Local variables are automatically created (allocated memory) when a function is called, and destroyed (deallocated) when the function terminates.

Global variables and global scope

A **global variable** is a variable that is defined outside of any function definition. Such variables are said to have **global scope**.

For example.

```
max_count = 1000    # a global variable
def funA(count):
    if count < max_count:
        ...
def funB(count):
    for i in range(max_count):
        ...
```

The use of global variables is generally considered to be bad programming style.

Documentation String (docstring)

[http://legacy.python.org/dev/peps/pep-0008/#documentation-](http://legacy.python.org/dev/peps/pep-0008/#documentation-strings)

A *docstring* is a string literal denoted by triple quotes given as the first line of certain program elements.

These additional lines must be indented at the same level.

The docstring for a module is simply the first block of quoted text to appear in the module.

Write docstrings for all public modules, functions, classes, and methods.

Convention:

```
"""Return a foobang
```

```
Optional plotz says to frobnicate the bizbaz  
first. """
```

Methods in Classes

- Define a method in a class by including function definitions within the scope of the class block
- There must be a special first argument **self** in all of method definitions which gets bound to the calling instance
- There is usually a special method called **__init__** in most classes

A Simple Class def: Student

```
class student:
    """A class representing a student """
    def __init__(self, n, a):
        self.full_name = n
        self.age = a
    def get_age(self):    #Method
        return self.age
```

- Define class:
 - Class name, begin with capital letter, by convention
 - object: class based on (Python built-in type)
- Define a method
 - Like defining a function
 - Must **have a special first parameter**, self, which provides way for a method to refer to object itself

Instantiating Objects with ‘__init__’

- `__init__` is the default constructor
- `__init__` serves as a constructor for the class. Usually does some initialization work
- An `__init__` method can take any number of arguments
- However, the first argument `self` in the definition of `__init__` is special

Self

- The first argument of every method is a reference to the current instance of the class
- By convention, we name this argument **self**
- In `__init__`, `self` refers to the object currently being created; so, in other class methods, it refers to the instance whose method was called
- Similar to the keyword `this` in Java or C++
- But Python uses `self` more often than Java uses `this`
- You **do not** give a value for this parameter(`self`) when you call the method, Python will provide it.

Public, Protected and Private Data

- If an identifier doesn't start with an underscore character "_" it can be accessed from outside, i.e. the value can be read and changed
- Data can be protected by making members private or protected. Instance variable names starting with two underscore characters cannot be accessed from outside of the class.
- At least not directly, but they can be accessed through private name mangling.
- That means, private data `__A` can be accessed by the following name construct: *`instance_name._classname__A`*

-
- If an identifier is only preceded by one underscore character, it is a protected member.
 - Protected members can be accessed like public members from outside of class

Example:

```
class Encapsulation(object):  
    def __init__(self, a, b, c):  
        self.public = a  
        self._protected = b  
        self.__private = c
```

The following table shows the different behavior Public, Protected and Private Data

Name	Notation	Behavior
name	Public	Can be accessed from inside and outside
_name	Protected	Like a public member, but they shouldn't be directly accessed from outside
__name	Private	Can't be seen and accessed from outside